Software Entwurf 4
Übung 1

HSD FH-HAGENBERG

Sommersemester 2026

Name:	Simon Offenberger	Abgabetermin:	KW 13
Mat.Nr.:	S2410306027	Punkte:	
Übungsgruppe:	1	korrigiert:	
Aufwand in Mh:	8		

Beispiel 1 (2+2 Punkte) Verständnisfrage: In einem Multithreaded-Programm haben Sie eine Funktion, wo (a) eine lokale Variable und alternativ dazu (b) eine statische lokale Variable deklariert ist. Erläutern Sie, ob es hier zu Problemen kommen kann - wenn ja, warum, wenn nein, warum nicht.

Beispiel 2 (4+1+2 Punkte) Main-Thread und Worker-Thread: (a) Schreiben Sie ein Multithreaded-Programm in C++, das neben dem Main-Thread weitere Worker-Threads hat. Die Worker-Threads rechnen jeweils die Fibonacci-Zahlen im Bereich 0 bis n und geben das Ergebnis auf der Konsole aus. Der Main-Thread übernimmt nur Organisatorisches (Threaderzeugen, Grenzwert n vorgeben, etc.). Die Anzahl der Worker-Threads setzen Sie dynamisch zur Laufzeit auf den Wert der vorhandenen logischen Kerne ihres Prozessors. Dazu verwenden Sie `std::thread::hardware_concurrency()` aus der C++-Thread-Library (`#include <thread>`). Bei Werten unter 4 setzen Sie die Threadanzahl zumindest auf 4.

Die Ausgabe besteht aus der ID des jeweiligen Threads und einem Text mit dem aktuellen Ergebnis:

```
WorkerThread 213 created successfully!
WorkerThread 257 created successfully!
WorkerThread 284 created successfully!
WorkerThread 292 created successfully!
Thread-ID 257: Result of fib(0) = 0
Thread-ID 213: Result of fib(0) = 0
Thread-ID 213: Result of fib(1) = 1
Thread-ID 257: Result of fib(1) = 1
Thread-ID 284: Result of fib(0) = 0
Thread-ID 292: Result of fib(0) = 0
Thread-ID 257: Result of fib(2) = 1
...
```

(b) Platzieren Sie zwischen der Ausgabe der Thread-ID, dem Fib-Ergebnis und an anderen Stellen die Anweisung `Sleep(0)` um Zeitverzögerungen zu simulieren. Testen Sie das Programm nun mehrmals und beobachten Sie die Ausgabe. Was können Sie zur Reihenfolge der Ausgaben und zur Reproduzierbarkeit sagen?

(c) Beschleunigung für ihren Chip mit k logischen Kernen: Rechnen Sie zuerst sequenziell k-Mal Fib(n) hintereinander in einem einzelnen Thread. Danach parallel Fib(n) mit k Workerthreads. Geben Sie die beiden Rechenzeiten aus und ermitteln Sie den Speedup.

Bsp für Dualcore (k=2): Sequenziell: Fib(n) Fib(n) Parallel: *Thread 1:* Fib(n) *Thread 2:* Fib(n)

Beispiel 3 (10+3 Punkte) Multithreaded Matrixmultiplikation: *(a)* Implementieren Sie eine multithreaded Matrixmultiplikation für eine quadratische Matrix der Dimension n in der Programmiersprache C++. Die Dimension ist eine beliebige aber fixe Konstante (Compilezeit). Die Matrizen enthalten ganze Zahlen. Verwenden Sie für jedes Element der Ergebnismatrix einen eigenen Thread. Überlegen Sie sich dabei, ob die Threads parallel (dh. „gleichzeitig") in die Ergebnismatrix schreiben dürfen.

Nachfolgend wird das Prinzip der Multiplikation anhand einer 3x3-Matrix gezeigt:

$$\begin{pmatrix} a11 & a12 & a13 \\ a21 & a22 & a23 \\ a31 & a32 & a33 \end{pmatrix} \begin{pmatrix} b11 & b12 & b13 \\ b21 & b22 & b23 \\ b31 & b32 & b33 \end{pmatrix} =$$

$$\begin{pmatrix} a11.b11 + a12.b21 + a13.b31 & a11.b12 + a12.b22 + a13.b32 & a11.b13 + a12.b23 + a13.b33 \\ a21.b11 + a22.b21 + a23.b31 & a21.b12 + a22.b22 + a23.b32 & a21.b13 + a22.b23 + a23.b33 \\ a31.b11 + a32.b21 + a33.b31 & a31.b12 + a32.b22 + a33.b32 & a31.b13 + a32.b23 + a33.b33 \end{pmatrix}$$

Bsp:

$$\begin{pmatrix} 1 & -2 & 3 \\ 4 & -5 & -6 \\ -7 & 8 & 9 \end{pmatrix} \mathbf{x} \begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 6 & 4 & 2 \\ -12 & -5 & 2 \\ 12 & 2 & -8 \end{pmatrix}$$

Testen Sie ihr Programm mit dem angeführten Beispiel und mit eigenen Eingaben. Geben Sie diese Tests auch ab.

(b) Beantworten Sie zudem folgende Fragen (mit Begründung):

(1) Kann es zu einer Racecondition kommen?
(2) Kann gleichzeitig zur Multiplikation ohne das Ergebnis zu verfälschen
 a) die Spur[1)] der Matrix A berechnet werden?
 b) die Matrix B transponiert[2)] werden?

Allgemeine Hinweise: Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine saubere Strukturierung und auf eine sorgfältige Ausarbeitung! Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab! Erstellen Sie eine **Funktionsbeschreibung** in LaTeX, in der Sie die Beantwortung der Fragen aufnehmen können!

[1)] Spur einer Matrix A: $Sp(A) = (a_{11}) + (a_{22}) + \ldots + (a_{nn}) = \sum_{i=1}^{n} (a_{ii})$

[2)] Eine transponierte Matrix A^T erhält man durch Austauschen von Spalten und Zeilen der Matrix A. Es gilt: $(a_{ij})^T = (a_{ji})$